Project AAC - BIOcean5D Coral Microbiomes: 02 - Data Pre-Processing

Eric J. Armstrong, eja98@cornell.edu, @EricJ_Armstrong

27 October, 2026

Contents

1 - Prepare Workspace **2**

1.1 - Define Paths 2

1.2 - Set Defaults 2

1.3 - Load Libraries 2

2 - Import Data **3**

2.1 - Load Phyloseq Data 3

3 - Data Cleanup **4**

3.1 - Remove Organelle Sequences 4

3.2 - Evaluate & Remove Controls 5

Final info for Reproducibility **13**

Session Information **13**

Purpose of this file This file contains all of the code for producing a curated OTU table from the 16S microbial sequence data for the Armstrong et al. manuscript entitled **"XXX"**, published by XXX.

Here, we will process the data into a phyloseq object.

```
ASV table
Taxonomy Table
Track Reads (metadata)
```

Then, we will remove the following:

```
Remove Chloroplasts
Remove Mitochondria.
Removing ASVs from negative controls and also negative control samples.
Evaluate accuracy by looking at the Mock Community.
Remove samples without "enough" reads.
```

Finally, write data file of phyloseq output, which will be stored in Data/02-AAC–PreProcessing/phyloseq_raw_prepro.RData.

If you have any questions, you are welcome to email the corresponding author at eja98@cornell.edu or tweet him at EricJ_Armstrong. Useful links: https://r-from-a-learners-perspective.readthedocs.io/en/latest/part5/

1 - Prepare Workspace

Load necessary R packages and set default parameters

1.1 - Define Paths

```
# a - Define the main project directory ----
  dir_proj <-
  "C:/Users/uax75/OneDrive/Documents/R/AAC-BIOcean5D-Coral_Microbiome/"

# b - Define the directory containing the input data ----
  dir_in <-
  "C:/Users/uax75/OneDrive/Documents/R/AAC-BIOcean5D-Coral_Microbiome/Data/"

# c - Define the directories that will contain output Files/Figures/Tables ----
  dir_out_dat <- paste(dir_in,
                       "02-AAC-PreProcessing/", sep = "")
  dir.create(dir_out_dat, recursive = TRUE)
  dir_out_fig <- paste(dir_proj,
                       "Figures/02-AAC-PreProcessing/", sep = "")
  dir.create(dir_out_fig, recursive = TRUE)
  dir_out_tab <- paste(dir_proj,
                       "Tables/02-AAC-PreProcessing/", sep = "")
  dir.create(dir_out_tab, recursive = TRUE)
```

1.2 - Set Defaults

```
# a - Define the output prefix for generated files ----
  pref_out <- "02-AAC-PreProcessing_"

# b - Set the random seed value ----
  set.seed(123)
```

1.3 - Load Libraries

```
# a - load necessary packages ----
  pacman::p_load(devtools, phyloseq, dada2, patchwork, xlsx, BiMiCo,
                 Biostrings, tidyverse, ggpubr, rstatix, ggplot2, expss,
                 decontam,
                 install = FALSE)
```

2 - Import Data

Load the necessary input data

2.1 - Load Phyloseq Data

```
# a - First, load the phyloseq object ----
  load(paste(dir_in,
             "01-AAC-AssignSeqVariants/",
             "01-AAC-AssignSeqVariants_physeq_raw.RData",
             sep = ""))

# b - Ceate a sample metadata table --------
  # Load the sample metadata table
  metadata <- read.xlsx(file = paste(dir_in,
                                     "01-AAC-AssignSeqVariants/",
                                     "Master_SampleKey.xlsx", sep = ""),
                        sheetName = "RData")

  # Extract the sample names from the physeq object to match to metadata
  sample.names <- colnames(otu_table(physeq_raw))

  # Reorganize the metadata table with the relevant data for the analysis
  samdf <- data.frame("Group" = vlookup(lookup_value = sample.names,
                                        dict = metadata,
                                        lookup_column = "FileName",
                                        result_column = "Group"),
                      "Site" = vlookup(lookup_value = sample.names,
                                        dict = metadata,
                                        lookup_column = "FileName",
                                        result_column = "Site"),
                      "Type" = vlookup(lookup_value = sample.names,
                                        dict = metadata,
                                        lookup_column = "FileName",
                                        result_column = "Type"),
                      "Subtype" = vlookup(lookup_value = sample.names,
                                        dict = metadata,
                                        lookup_column = "FileName",
                                        result_column = "Subtype"),
                      "Transplant" = vlookup(lookup_value = sample.names,
                                        dict = metadata,
                                        lookup_column = "FileName",
                                        result_column = "Transplant"),
                      "Treatment" = vlookup(lookup_value = sample.names,
                                        dict = metadata,
                                        lookup_column = "FileName",
                                        result_column = "Treatment"),
                      "Colony" = vlookup(lookup_value = sample.names,
                                        dict = metadata,
                                        lookup_column = "FileName",
                                        result_column = "Colony"),
                      "Host" = vlookup(lookup_value = sample.names,
```

```
                                    dict = metadata,
                                    lookup_column = "FileName",
                                    result_column = "Species"))

  head(samdf)
#>        Group    Site  Type   Subtype      Transplant Treatment Colony Host
#> 1 TrueSample Tiahura Coral Coral POC        Genotype  Genotype    C01  POC
#> 2 TrueSample Opunohu Coral Coral POC        Genotype  Genotype    C05  POC
#> 3 TrueSample Tiahura Coral Coral POC Back Transplant    Pre-HS    C01  POC
#> 4 TrueSample Tiahura Coral Coral POC Back Transplant    Pre-HS    C02  POC
#> 5 TrueSample Tiahura Coral Coral POC Back Transplant    Pre-HS    C03  POC
#> 6 TrueSample Tiahura Coral Coral POC Back Transplant    Pre-HS    C04  POC
  row.names(samdf) <- sample.names

# d - Append the sample metadata to the Phyloseq object ----
  sample_data(physeq_raw) <- samdf

  # Turn the 'treatment' column into a character vector and reorder
  sample_data(physeq_raw)$Treatment <-
    as.character(sample_data(physeq_raw)$Treatment)

  sample_data(physeq_raw)$Treatment <-
    factor(sample_data(physeq_raw)$Treatment,
           levels=c("Genotype", "2-Weeks",
                    "4-Months", "Pre-HS", "Post-HS", NA))

  # Add a column with the SampleID names to the Phyloseq object
  sample_data(physeq_raw)['SampleID'] <- row.names(sample_data(physeq_raw))
```

3 - Data Cleanup

3.1 - Remove Organelle Sequences

```
# a - Remove Chloroplast Sequences ----

  # Make a new physeq without chloroplasts
    physeq_noChloros <-
      physeq_raw %>%
      # rm chloroplasts
      subset_taxa(Order != "Chloroplast" | is.na(Order))

  # How many taxa were chloroplasts?
    num_chloro_ASVs <- ntaxa(physeq_raw) - ntaxa(physeq_noChloros)
    num_chloro_ASVs
#> [1] 487

# b - Remove Mitochondrial Sequences ----

  # Make a new physeq without chloroplasts
    physeq_noChlorosMitos <-
      physeq_noChloros %>%
```

```
      subset_taxa(Family != "Mitochondria" | is.na(Family))

    # How many mitochondrial ASVs?
      num_mito_ASVs <- ntaxa(physeq_noChloros) - ntaxa(physeq_noChlorosMitos)
      num_mito_ASVs
#> [1] 94

# c - Total number of ASVs removed from chloros and mitos
  ntaxa(physeq_raw) - ntaxa(physeq_noChlorosMitos)
#> [1] 581

# d - Proportion of ASVs kept?
  ntaxa(physeq_noChlorosMitos)/ntaxa(physeq_raw)
#> [1] 0.9597031
```

3.2 - Evaluate & Remove Controls

Inspect the negative controls, then decide which of the ASVs found in them are genuine contaminants rather than real taxa carried into the blanks by index hopping and low-level cross-contamination.

```
Negative controls
ASVs found within the negative controls and their distribution in the samples.
Statistical identification of contaminants with decontam
```

Four negative controls carry reads - one Field Blank, one PCR Blank and two Extraction Blanks. (A fifth control in the sample key yielded none, and script 01 drops `neg-pcr2` as a bad sample.)

This step changed on 27 October 2026. Two faults had to be corrected together: `ASVs_control` was built as a one-column data.frame rather than a character vector, so the removal in step i matched nothing and no ASV was ever discarded; and until the environmental and control FASTQs were re-trimmed to the common amplicon window (see section 3.2 of script 01), no blank ASV could have matched a coral ASV in any case, because the two sets of reads began at different bases. With both repaired, the blanks share an ASV space with the samples for the first time and this filter finally has real work to do - which is also why blanket removal of every blank ASV is no longer acceptable and decontam is used instead.

```
# a - Create a vector of samples that were negative controls --
  samples_control <-
  rownames(subset(sample_data(physeq_raw),
                  Subtype %in%
                    c("PCR Blank", "Extraction Blank", "Field Blank")))

# b - Make a new phyloseq object of only the control samples ----
  physeq_control <-
    physeq_noChlorosMitos %>%
    prune_samples(
      (sample_names(physeq_noChlorosMitos) %in% samples_control), .) %>%
    #subset_samples(., names %in% samples_control) %>%
    # force remove ASVs with 0 counts
    prune_taxa(taxa_sums(.) > 0, .)

  physeq_control
#> phyloseq-class experiment-level object
```

```
#> otu_table()   OTU Table:         [ 164 taxa and 4 samples ]
#> sample_data() Sample Data:       [ 4 samples by 9 sample variables ]
#> tax_table()   Taxonomy Table:    [ 164 taxa by 10 taxonomic ranks ]
#> refseq()      DNAStringSet:      [ 164 reference sequences ]

# c - Control ASVs, in both identifier namespaces ----
  # NOTE: as.vector() does NOT flatten a data.frame - it returns it unchanged -
  # so the previous version of this object was a one-column data.frame. Step e
  # worked because it indexed ASVs_control$ASV, but step i tested
  # `ASV %in% ASVs_control` against the data.frame itself, which coerces to a
  # deparsed string, matches nothing, and silently removed no taxa at all.
  # dplyr::pull() returns the character vector both steps assume.
  ASVs_control <-
    physeq_control %>%
    tax_table() %>%
    data.frame() %>%
    dplyr::pull(ASV)

  # This object carries the tax_table's ASV column ("ASV_1", "ASV_2", ...),
  # which is NOT the same namespace as taxa_names(): script 01 renames taxa via
  # microbiome::add_refseq(tag = "ASV") to "ASV1", "ASV2", .... Pruning must use
  # taxa_names(), so keep a matching vector in that namespace and never mix them.
  seqs_control <- taxa_names(physeq_control)

  stopifnot(length(ASVs_control) == length(seqs_control))

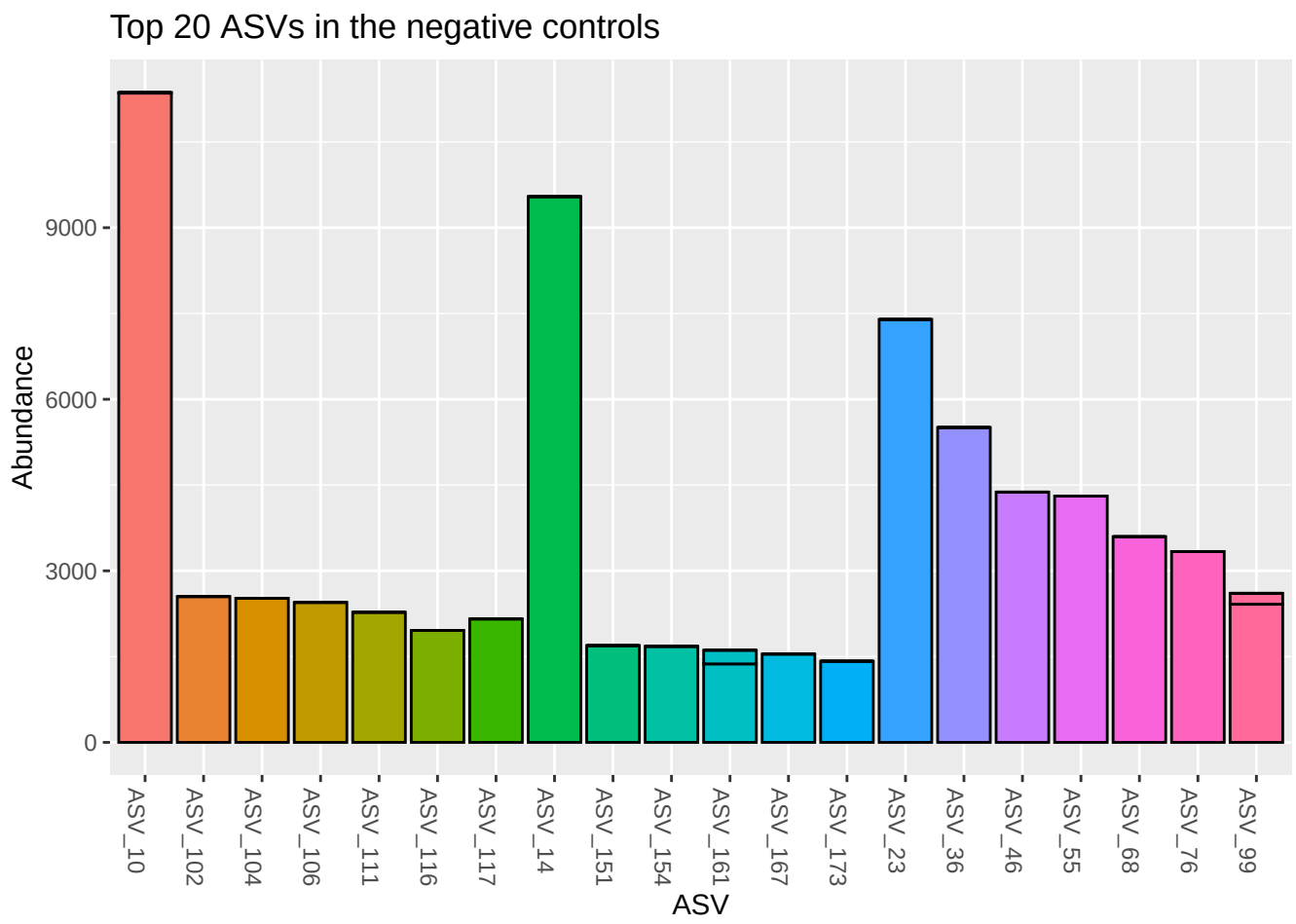
# c2 - Cap the set used for plotting and testing ----
  # Steps d-h draw one facet panel and one Wilcoxon test per blank ASV. Before
  # the amplicon window was repaired the blanks shared almost nothing with the
  # samples, so that was a handful of panels; now that they occupy the same ASV
  # space it can be hundreds, which makes the figure unreadable and risks
  # stat_pvalue_manual() failing on NA positions from constant-value groups.
  # The diagnostic is therefore capped at the most abundant blank ASVs, ranked
  # by their abundance IN THE CONTROLS - the ones that actually characterise the
  # blanks. Removal in steps i-k still considers every ASV, so nothing is
  # excluded from the filtering itself; this cap is presentational only.
  n_plot_control_asvs <- 20

  seqs_control_top <-
    physeq_control %>%
    taxa_sums() %>%
    sort(decreasing = TRUE) %>%
    names() %>%
    head(n_plot_control_asvs)

  cat("Blank ASVs:", length(seqs_control), "total;",
      length(seqs_control_top), "shown in the diagnostic plots.\n")
#> Blank ASVs: 164 total; 20 shown in the diagnostic plots.

# d - Visualize raw abundance values of control ASVs ----
  plot_bar(prune_taxa(seqs_control_top, physeq_control),
           "ASV", fill = "ASV") +
    labs(title = paste0("Top ", length(seqs_control_top),
```

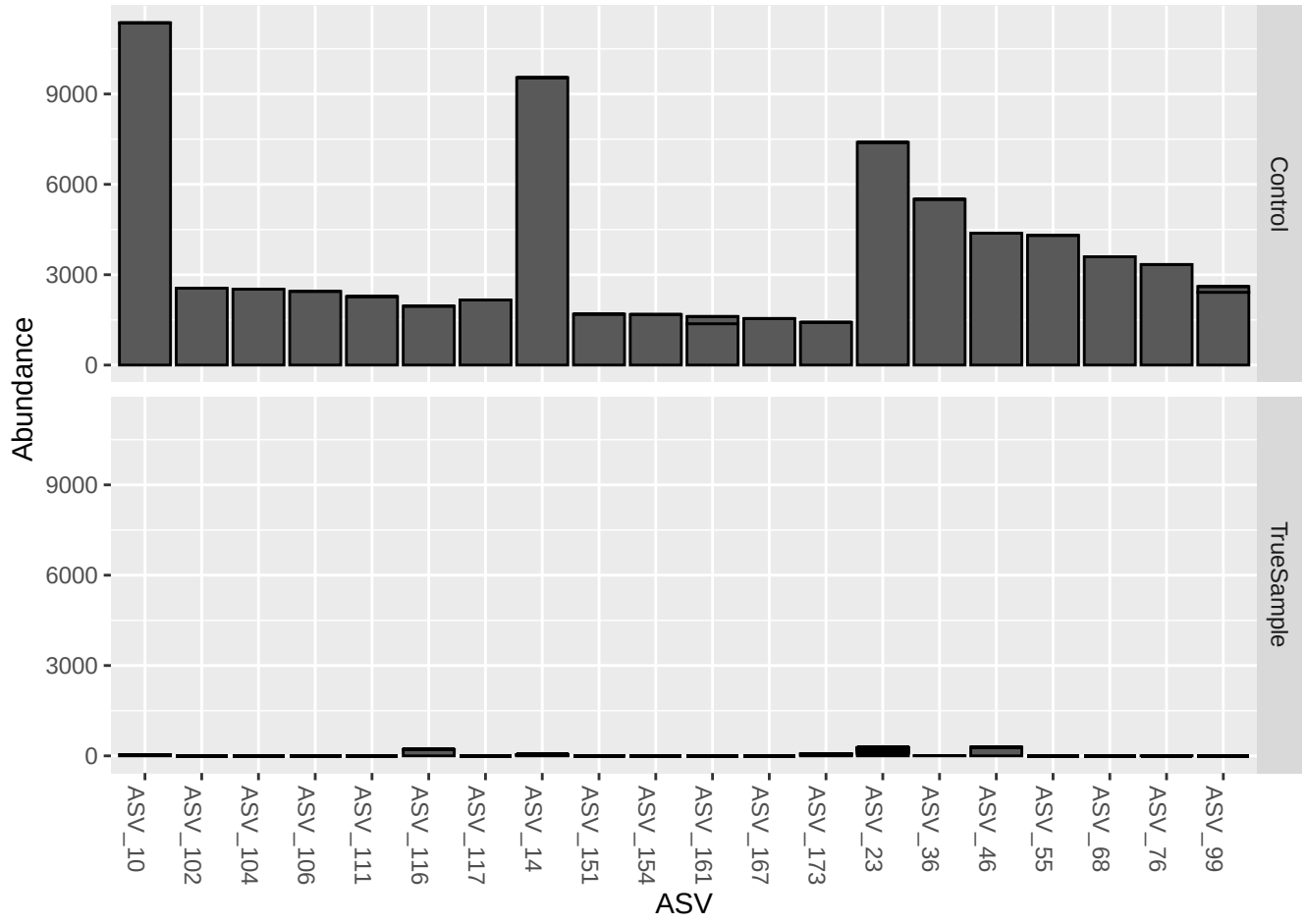
```
                         " ASVs in the negative controls")) +
    theme(legend.position = "none")
```




```
# e - Use vector to subset from larger dataset ----
  # prune_taxa() rather than subset_taxa(ASV %in% ...) so this stays in the
  # taxa_names() namespace and cannot be crossed with the tax_table ASV column.
  physeq_neg_controlASV <-
    physeq_noChlorosMitos %>%
    prune_taxa(seqs_control_top, .) %>%
    # we will also need to remove the positive controls from the samples!
    # We will evaluate the Mock community a bit later on. :)
    subset_samples(., Subtype != "ZymoMock")

  physeq_neg_controlASV
#> phyloseq-class experiment-level object
#> otu_table()   OTU Table:         [ 20 taxa and 157 samples ]
#> sample_data() Sample Data:       [ 157 samples by 9 sample variables ]
#> tax_table()   Taxonomy Table:    [ 20 taxa by 10 taxonomic ranks ]
#> refseq()      DNAStringSet:      [ 20 reference sequences ]
```

```
# f - Plot Abundances of neg control ASVs in samples versus controls ----
  plot_bar(physeq_neg_controlASV, "ASV", facet_grid = Group~.) +
    theme(legend.position = "none")
```




```
# g - Calculate the phylum relative abundance
  # Note: The read depth MUST be normalized in some way: scale_reads
  asv_negControls_df <-
    physeq_neg_controlASV %>%
    # agglomerate at the ASV level
    tax_glom(taxrank = "ASV") %>%
    # Melt to a long format
    psmelt()

  # Set levels of Group variable so that "Control" comes first
  asv_negControls_df$Group <- factor(asv_negControls_df$Group,
                                     levels = c("Control", "TrueSample"))

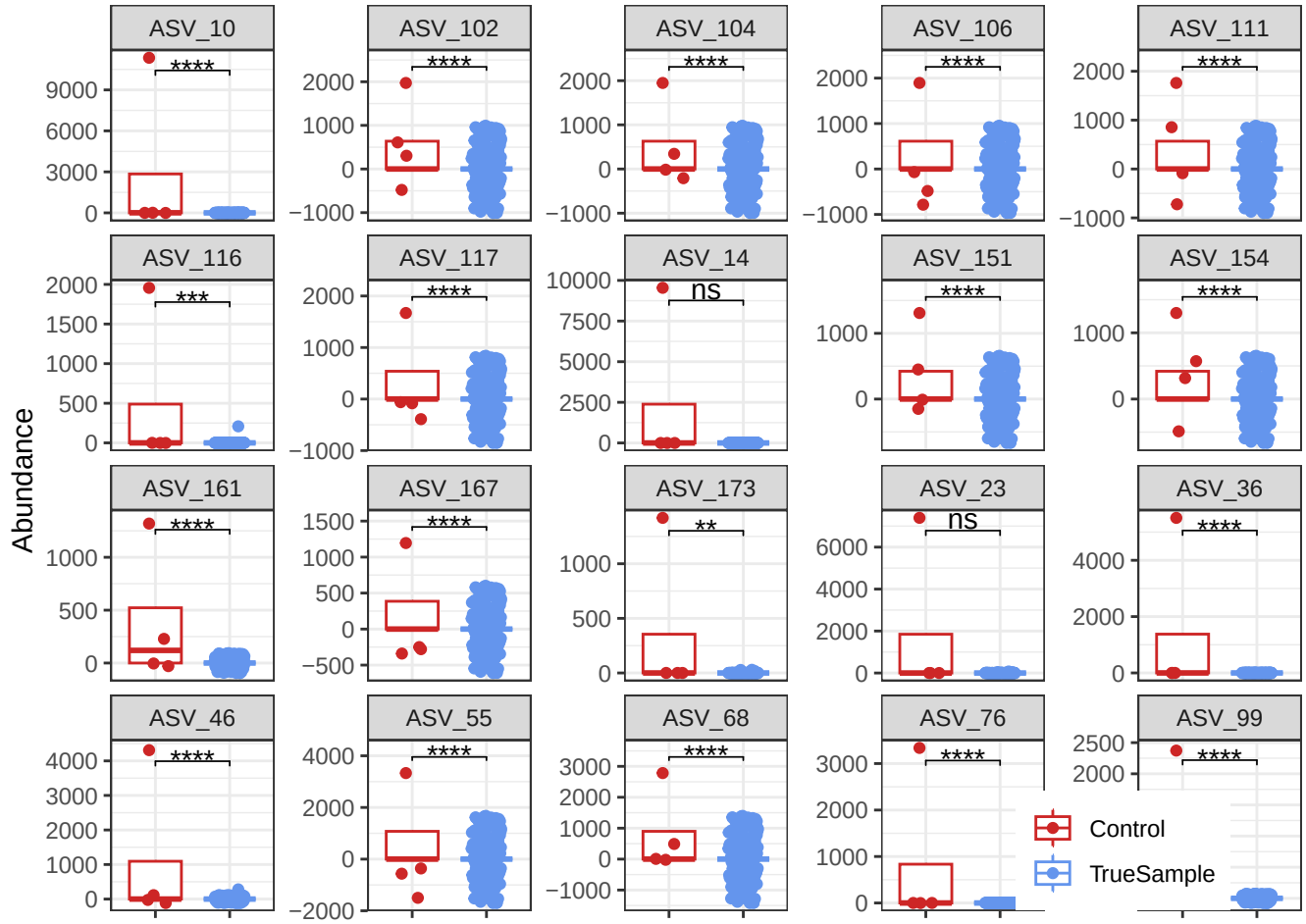
  # Check
  head(asv_negControls_df)
#>        OTU                       Sample Abundance   Group Site    Type
```

```
#> 131  ASV10    NC-PCR-NegControlPCRBlank     11353 Control <NA> Control
#> 1202 ASV14        NC2-NegControlSoilKit      9545 Control <NA> Control
#> 2135 ASV23        NC2-NegControlSoilKit      7396 Control <NA> Control
#> 2308 ASV36                       60-NC      5507 Control <NA> Control
#> 2432 ASV46                       60-NC      4377 Control <NA> Control
#> 2661 ASV55 NC3-NegControlExtractionKit2     4307 Control <NA> Control
#>              Subtype Transplant Treatment Colony Host
#> 131         PCR Blank       <NA>      <NA>   <NA> <NA>
#> 1202 Extraction Blank       <NA>      <NA>   <NA> <NA>
#> 2135 Extraction Blank       <NA>      <NA>   <NA> <NA>
#> 2308      Field Blank       <NA>      <NA>   <NA> <NA>
#> 2432      Field Blank       <NA>      <NA>   <NA> <NA>
#> 2661 Extraction Blank       <NA>      <NA>   <NA> <NA>
#>                          SampleID   Kingdom         Phylum               Class
#> 131     NC-PCR-NegControlPCRBlank Eukaryota           <NA>                <NA>
#> 1202        NC2-NegControlSoilKit  Bacteria Actinomycetota      Actinobacteria
#> 2135        NC2-NegControlSoilKit  Bacteria      Bacillota             Bacilli
#> 2308                       60-NC  Bacteria Pseudomonadota Alphaproteobacteria
#> 2432                       60-NC  Bacteria   Bacteroidota         Bacteroidia
#> 2661 NC3-NegControlExtractionKit2  Bacteria Pseudomonadota Alphaproteobacteria
#>                    Order               Family             Genus Species
#> 131                 <NA>                 <NA>              <NA>    <NA>
#> 1202 Propionibacteriales Propionibacteriaceae     Cutibacterium   acnes
#> 2135    Staphylococcales    Staphylococcaceae    Staphylococcus  aureus
#> 2308       Reyranellales       Reyranellaceae        Reyranella    <NA>
#> 2432     Chitinophagales     Chitinophagaceae Sediminibacterium    <NA>
#> 2661    Sphingomonadales    Sphingomonadaceae       Sphingobium amiense
#>      Subspecies    ASV
#> 131        <NA> ASV_10
#> 1202       <NA> ASV_14
#> 2135       <NA> ASV_23
#> 2308       <NA> ASV_36
#> 2432  salmoneum ASV_46
#> 2661       <NA> ASV_55

# h - Statistically test!
  library(rstatix)
  negControl_ASV_statTest <-
    asv_negControls_df %>%
    group_by(ASV) %>%
    wilcox_test(Abundance ~ Group) %>%
    # Add the x and y positions for plotting
    adjust_pvalue(method = "fdr") %>%
    # Make the significance values be stars for plotting
    add_significance() %>%
    # Calculate where to place it on the facet grid plot
    add_xy_position(x = "Group", fun = "max", scales = "free") %>%
    # bring the pvalues a lil lower
    mutate(y.position = y.position * 0.82)

  ## Using ggpubr
  ggboxplot(
    asv_negControls_df,
```

```
    x = "Group", y = "Abundance",
    color = "Group",
    facet.by = "ASV", scales = "free_y",add = "jitter") +
    stat_pvalue_manual(negControl_ASV_statTest) +
    scale_color_manual(values = c("firebrick3", "cornflowerblue")) +
    scale_fill_manual(values = c("firebrick3", "cornflowerblue")) +
    theme_bw() +
    theme(axis.text.x = element_blank(), axis.title.x = element_blank(),
          legend.title = element_blank(), legend.position = c(0.85, 0.07))
```




```
# i - Identify contaminants with decontam ----
  # Removing every ASV that appears in a blank is too blunt. The controls share
  # the 515F/926R window with the corals, so index hopping and low-level
  # carry-over place genuine, abundant taxa in the blanks; blanket removal would
  # delete real community members. decontam's prevalence model asks the right
  # question instead - is this ASV MORE prevalent in the controls than in the
  # true samples? - and flags only those.
  #
  # threshold = 0.5 implements the "more prevalent in controls than in samples"
  # rule. It is the appropriate choice here because only four negative controls
```

```
  # carry reads (one Field, one PCR, two Extraction blanks; the fifth control in
  # the sample key yielded none), which leaves the default threshold of 0.1 with
  # very little power.

  sample_data(physeq_noChlorosMitos)$is_control <-
    sample_data(physeq_noChlorosMitos)$Group == "Control"

  contam_df <-
    decontam::isContaminant(physeq_noChlorosMitos,
                            method    = "prevalence",
                            neg       = "is_control",
                            threshold = 0.5)

  seqs_contaminant <- rownames(contam_df)[contam_df$contaminant]

# j - Report what is removed, against what blanket removal would have done ----
  physeq_true <-
    physeq_noChlorosMitos %>%
    subset_samples(., Group == "TrueSample") %>%
    prune_taxa(taxa_sums(.) > 0, .)

  pct_reads <- function(ps, taxa) {
    keep <- intersect(taxa_names(ps), taxa)
    if (length(keep) == 0) return(0)
    100 * sum(otu_table(prune_taxa(keep, ps))) / sum(otu_table(ps))
  }

  cat("\n--- Control filtering ---\n",
      "ASVs detected in the blanks   :", length(seqs_control),
      sprintf("(%.2f%% of true-sample reads)", pct_reads(physeq_true, seqs_control)), "\n",
      "ASVs flagged by decontam      :", length(seqs_contaminant),
      sprintf("(%.2f%% of true-sample reads)", pct_reads(physeq_true, seqs_contaminant)), "\n",
      "Blank ASVs retained as real   :",
      length(setdiff(seqs_control, seqs_contaminant)), "\n\n")
#>
#> --- Control filtering ---
#>  ASVs detected in the blanks   : 164 (0.22% of true-sample reads)
#>  ASVs flagged by decontam      : 25 (0.22% of true-sample reads)
#>  Blank ASVs retained as real   : 139

  # Taxonomy of what is being discarded, for the record
  if (length(seqs_contaminant) > 0) {
    physeq_noChlorosMitos %>%
      prune_taxa(seqs_contaminant, .) %>%
      tax_table() %>%
      data.frame() %>%
      dplyr::select(Phylum, Class, Order, Family, Genus, ASV) %>%
      print()
  }
#>                Phylum              Class                            Order
#> ASV10             <NA>               <NA>                             <NA>
#> ASV14   Actinomycetota      Actinobacteria                Propionibacteriales
#> ASV23        Bacillota             Bacilli                   Staphylococcales
```

```
#> ASV36   Pseudomonadota Alphaproteobacteria                      Reyranellales
#> ASV46     Bacteroidota         Bacteroidia                     Chitinophagales
#> ASV76   Actinomycetota      Actinobacteria                     Mycobacteriales
#> ASV99   Pseudomonadota Alphaproteobacteria                    Sphingomonadales
#> ASV108  Pseudomonadota Gammaproteobacteria                     Pseudomonadales
#> ASV116  Pseudomonadota Alphaproteobacteria                    Sphingomonadales
#> ASV161  Actinomycetota      Actinobacteria                       Micrococcales
#> ASV173  Pseudomonadota Gammaproteobacteria                     Pseudomonadales
#> ASV234  Pseudomonadota Alphaproteobacteria                     Acetobacterales
#> ASV262  Pseudomonadota Gammaproteobacteria                     Pseudomonadales
#> ASV285  Pseudomonadota Gammaproteobacteria Gammaproteobacteria Incertae Sedis
#> ASV290  Pseudomonadota Gammaproteobacteria                     Pseudomonadales
#> ASV398  Halobacteriota        Halobacteria                      Halobacterales
#> ASV412  Pseudomonadota Gammaproteobacteria                     Pseudomonadales
#> ASV414  Halobacteriota        Halobacteria                      Halobacterales
#> ASV541  Actinomycetota      Actinobacteria                       Micrococcales
#> ASV573  Pseudomonadota Gammaproteobacteria                    Enterobacterales
#> ASV1068   Bacteroidota         Bacteroidia                     Chitinophagales
#> ASV1397      Bacillota             Bacilli                    Staphylococcales
#> ASV1595 Pseudomonadota Gammaproteobacteria                     Burkholderiales
#> ASV3291 Pseudomonadota Gammaproteobacteria                     Pseudomonadales
#> ASV3919           <NA>                <NA>                                <NA>
#>                      Family             Genus      ASV
#> ASV10                  <NA>              <NA>   ASV_10
#> ASV14   Propionibacteriaceae     Cutibacterium   ASV_14
#> ASV23      Staphylococcaceae    Staphylococcus   ASV_23
#> ASV36         Reyranellaceae        Reyranella   ASV_36
#> ASV46       Chitinophagaceae Sediminibacterium   ASV_46
#> ASV76     Corynebacteriaceae        Lawsonella   ASV_76
#> ASV99      Sphingomonadaceae      Sphingomonas   ASV_99
#> ASV108      Pseudomonadaceae       Pseudomonas  ASV_108
#> ASV116     Sphingomonadaceae      Sphingomonas  ASV_116
#> ASV161        Micrococcaceae           Kocuria  ASV_161
#> ASV173         Moraxellaceae     Acinetobacter  ASV_173
#> ASV234      Acetobacteraceae         Acidisoma  ASV_234
#> ASV262         Moraxellaceae     Acinetobacter  ASV_262
#> ASV285        Unknown Family       Acidibacter  ASV_285
#> ASV290      Pseudomonadaceae       Pseudomonas  ASV_290
#> ASV398      Halomicrobiaceae        Halomarina  ASV_398
#> ASV412         Moraxellaceae     Acinetobacter  ASV_412
#> ASV414         Halococcaceae        Halococcus  ASV_414
#> ASV541     Brevibacteriaceae    Brevibacterium  ASV_541
#> ASV573           Erwiniaceae           Pantoea  ASV_573
#> ASV1068     Chitinophagaceae     Vibrionimonas ASV_1068
#> ASV1397    Staphylococcaceae    Staphylococcus ASV_1397
#> ASV1595       Comamonadaceae       Brachymonas ASV_1595
#> ASV3291        Moraxellaceae     Acinetobacter ASV_3291
#> ASV3919                <NA>              <NA> ASV_3919

# k - Remove contaminant taxa and the control samples ----
  keep_taxa <- setdiff(taxa_names(physeq_noChlorosMitos), seqs_contaminant)

  physeq_noControlsChlorosMitos <-
```

```
    physeq_noChlorosMitos %>%
    prune_taxa(keep_taxa, .) %>%
    # Remove control samples
    subset_samples(., !(Subtype %in%
                          c("PCR Blank", "Extraction Blank", "Field Blank"))) %>%
    # Drop ASVs left at zero once the controls are gone
    prune_taxa(taxa_sums(.) > 0, .)

  physeq_noControlsChlorosMitos
#> phyloseq-class experiment-level object
#> otu_table()   OTU Table:         [ 13673 taxa and 153 samples ]
#> sample_data() Sample Data:       [ 153 samples by 10 sample variables ]
#> tax_table()   Taxonomy Table:    [ 13673 taxa by 10 taxonomic ranks ]
#> refseq()      DNAStringSet:      [ 13673 reference sequences ]


# j - save processed physeq object ----
  save(physeq_noControlsChlorosMitos,
         file = paste(dir_out_dat,
                      pref_out,
                      "physeq_noControlsChlorosMitos.RData", sep = ""))
```

Final info for Reproducibility

Check Render Time

```
# Take the time now that we are at the end of the script
end_time <- Sys.time()
end_time
#> [1] "2026-10-27 18:21:45 EDT"

# Echo the elapsed time
elapsed_time <- round((end_time - start_time), 3)
elapsed_time
#> Time difference of 33.813 secs
```

Session Information

```
# Ensure reproducibility with package version information
devtools::session_info()
#> - Session info ---------------------------------------------------------------
#>  setting  value
#>  version  R version 4.6.1 (2026-06-24 ucrt)
#>  os       Windows 11 x64 (build 26300)
#>  system   x86_64, mingw32
#>  ui       RTerm
#>  language (EN)
#>  collate  English_United States.utf8
#>  ctype    English_United States.utf8
```

```
#>  tz       America/New_York
#>  date     2026-10-27
#>  pandoc   3.6.3 @ C:/Program Files/RStudio/resources/app/bin/quarto/bin/tools/ (via rmarkdown)
#>  quarto   1.7.32 @ C:\\PROGRA~1\\RStudio\\RESOUR~1\\app\\bin\\quarto\\bin\\quarto.exe
#>
#> - Packages -------------------------------------------------------------------
#>  ! package              * version  date (UTC) lib source
#>    abind                  1.4-8    2024-09-12 [1] CRAN (R 4.6.1)
#>    ade4                   1.7-24   2026-03-21 [1] CRAN (R 4.6.1)
#>    ape                    5.8-1    2024-12-16 [1] CRAN (R 4.6.1)
#>    backports              1.5.1    2026-04-03 [1] CRAN (R 4.6.1)
#>    BiMiCo               * 0.1.0    2026-09-18 [1] Github (peterolah001/BiMiCo@01a80b4)
#>    Biobase                2.72.0   2026-04-28 [1] https://bioc-release.r-universe.dev (R 4.6.0)
#>    BiocGenerics         * 0.58.1   2026-05-14 [1] https://bioc-release.r-universe.dev (R 4.6.0)
#>    BiocParallel           1.46.0   2026-04-28 [1] https://bioc-release.r-universe.dev (R 4.6.0)
#>    biomformat             1.40.0   2026-04-28 [1] https://bioc-release.r-universe.dev (R 4.6.0)
#>    Biostrings           * 2.80.2   2026-09-02 [1] Bioconductor 3.23 (R 4.6.1)
#>    bitops                 1.1-0    2026-07-30 [1] CRAN (R 4.6.1)
#>    broom                  1.0.13   2026-05-14 [1] CRAN (R 4.6.1)
#>    cachem                 1.1.0    2024-05-16 [1] CRAN (R 4.6.1)
#>    car                    3.1-5    2026-02-03 [1] CRAN (R 4.6.1)
#>    carData                3.0-6    2026-01-30 [1] CRAN (R 4.6.1)
#>    checkmate              2.3.4    2026-02-03 [1] CRAN (R 4.6.1)
#>    cigarillo              1.2.1    2026-07-13 [1] Bioconductor 3.23 (R 4.6.1)
#>    cli                    3.6.6    2026-04-09 [1] CRAN (R 4.6.1)
#>    cluster                2.1.8.2  2026-02-05 [2] CRAN (R 4.6.1)
#>    codetools              0.2-20   2024-03-31 [2] CRAN (R 4.6.1)
#>    crayon                 1.5.3    2024-06-20 [1] CRAN (R 4.6.1)
#>    dada2                * 1.40.0   2026-04-28 [1] https://bioc-release.r-universe.dev (R 4.6.0)
#>    data.table             1.18.6.1 2026-08-24 [1] CRAN (R 4.6.1)
#>    decontam             * 1.32.0   2026-04-28 [1] https://bioc-release.r-universe.dev (R 4.6.0)
#>    DelayedArray           0.38.2   2026-05-26 [1] https://bioc-release.r-universe.dev (R 4.6.0)
#>    deldir                 2.0-4    2024-02-28 [1] CRAN (R 4.6.1)
#>    devtools             * 2.5.2    2026-04-30 [1] CRAN (R 4.6.1)
#>    dichromat              2.0-1    2026-07-22 [1] CRAN (R 4.6.1)
#>    digest                 0.6.39   2025-11-19 [1] CRAN (R 4.6.1)
#>    dplyr                * 1.2.1    2026-04-03 [1] CRAN (R 4.6.1)
#>    ellipsis               0.3.3    2026-04-04 [1] CRAN (R 4.6.1)
#>    evaluate               1.0.5    2025-08-27 [1] CRAN (R 4.6.1)
#>    expss                * 0.11.7   2025-10-15 [1] CRAN (R 4.6.1)
#>    farver                 2.1.2    2024-05-13 [1] CRAN (R 4.6.1)
#>    fastmap                1.2.0    2024-05-15 [1] CRAN (R 4.6.1)
#>    forcats              * 1.0.1    2025-09-25 [1] CRAN (R 4.6.1)
#>    foreach                1.5.2    2022-02-02 [1] CRAN (R 4.6.1)
#>    Formula                1.2-6    2026-08-03 [1] CRAN (R 4.6.1)
#>    fs                     2.1.0    2026-04-18 [1] CRAN (R 4.6.1)
#>    generics             * 0.1.4    2025-05-09 [1] CRAN (R 4.6.1)
#>    GenomicAlignments      1.48.0   2026-04-28 [1] https://bioc-release.r-universe.dev (R 4.6.0)
#>    GenomicRanges          1.64.0   2026-04-28 [1] https://bioc-release.r-universe.dev (R 4.6.0)
#>    ggplot2              * 4.0.3    2026-04-22 [1] CRAN (R 4.6.1)
#>    ggpubr               * 1.0.0    2026-07-06 [1] CRAN (R 4.6.1)
#>    ggsignif               0.6.4    2022-10-13 [1] CRAN (R 4.6.1)
#>    glue                   1.8.1    2026-04-17 [1] CRAN (R 4.6.1)
```

```
#>    gtable                0.3.6    2024-10-25 [1] CRAN (R 4.6.1)
#>    hms                   1.1.4    2025-10-17 [1] CRAN (R 4.6.1)
#>    htmlTable             2.5.0    2026-04-22 [1] CRAN (R 4.6.1)
#>    htmltools             0.5.9    2025-12-04 [1] CRAN (R 4.6.1)
#>    htmlwidgets           1.6.4    2023-12-06 [1] CRAN (R 4.6.1)
#>    hwriter               1.3.2.1  2022-04-08 [1] CRAN (R 4.6.1)
#>    igraph                2.3.3    2026-06-26 [1] CRAN (R 4.6.1)
#>    interp                1.1-6    2024-01-26 [1] CRAN (R 4.6.1)
#>    IRanges             * 2.46.0   2026-04-28 [1] https://bioc-release.r-universe.dev (R 4.6.0)
#>    iterators             1.0.14   2022-02-05 [1] CRAN (R 4.6.1)
#>    jpeg                  0.1-11   2025-03-21 [1] CRAN (R 4.6.1)
#>    jsonlite              2.0.0    2025-03-27 [1] CRAN (R 4.6.1)
#>    knitr                 1.52     2026-09-06 [1] CRAN (R 4.6.1)
#>    labeling              0.4.3    2023-08-29 [1] CRAN (R 4.6.1)
#>    lattice               0.22-9   2026-02-09 [2] CRAN (R 4.6.1)
#>    latticeExtra          0.6-31   2025-09-10 [1] CRAN (R 4.6.1)
#>    lifecycle             1.0.5    2026-01-08 [1] CRAN (R 4.6.1)
#>    lubridate           * 1.9.5    2026-02-04 [1] CRAN (R 4.6.1)
#>    maditr              * 0.8.7    2026-01-19 [1] CRAN (R 4.6.1)
#>    magrittr              2.0.5    2026-04-04 [1] CRAN (R 4.6.1)
#>    MASS                  7.3-65   2025-02-28 [2] CRAN (R 4.6.1)
#>    Matrix                1.7-5    2026-03-21 [2] CRAN (R 4.6.1)
#>    MatrixGenerics        1.24.0   2026-04-28 [1] https://bioc-release.r-universe.dev (R 4.6.0)
#>    matrixStats           1.5.0    2025-01-07 [1] CRAN (R 4.6.1)
#>    memoise               2.0.1    2021-11-26 [1] CRAN (R 4.6.1)
#>    mgcv                  1.9-4    2025-11-07 [2] CRAN (R 4.6.1)
#>    multtest              2.68.0   2026-04-28 [1] https://bioc-release.r-universe.dev (R 4.6.0)
#>    nlme                  3.1-169  2026-03-27 [2] CRAN (R 4.6.1)
#>    otel                  0.2.0    2025-08-29 [1] CRAN (R 4.6.1)
#>    pacman                0.5.1    2019-03-11 [1] CRAN (R 4.6.1)
#>    patchwork           * 1.3.2    2025-08-25 [1] CRAN (R 4.6.1)
#>    permute               0.9-10   2026-02-06 [1] CRAN (R 4.6.1)
#>    phyloseq            * 1.56.0   2026-04-28 [1] https://bioc-release.r-universe.dev (R 4.6.0)
#>    pillar                1.11.1   2025-09-17 [1] CRAN (R 4.6.1)
#>    pkgbuild              1.4.8    2025-05-26 [1] CRAN (R 4.6.1)
#>    pkgconfig             2.0.3    2019-09-22 [1] CRAN (R 4.6.1)
#>    pkgload               1.5.3    2026-06-15 [1] CRAN (R 4.6.1)
#>    plyr                  1.8.9    2023-10-02 [1] CRAN (R 4.6.1)
#>    png                   0.1-9    2026-03-15 [1] CRAN (R 4.6.1)
#>    purrr               * 1.2.2    2026-04-10 [1] CRAN (R 4.6.1)
#>    pwalign               1.8.0    2026-04-28 [1] https://bioc-release.r-universe.dev (R 4.6.0)
#>    R6                    2.6.1    2025-02-15 [1] CRAN (R 4.6.1)
#>    RColorBrewer          1.1-3    2022-04-03 [1] CRAN (R 4.6.1)
#>    Rcpp                * 1.1.2    2026-07-05 [1] CRAN (R 4.6.1)
#>  D RcppParallel          6.2.1    2026-08-27 [1] CRAN (R 4.6.1)
#>    readr               * 2.2.0    2026-02-19 [1] CRAN (R 4.6.1)
#>    reshape2              1.4.5    2025-11-12 [1] CRAN (R 4.6.1)
#>    rJava                 1.0-18   2026-04-08 [1] CRAN (R 4.6.1)
#>    rlang                 1.3.0    2026-07-05 [1] CRAN (R 4.6.1)
#>    rmarkdown             2.32     2026-09-01 [1] CRAN (R 4.6.1)
#>    Rsamtools             2.28.0   2026-04-28 [1] https://bioc-release.r-universe.dev (R 4.6.0)
#>    rstatix             * 1.1.0    2026-07-23 [1] CRAN (R 4.6.1)
#>    rstudioapi            0.19.0   2026-06-11 [1] CRAN (R 4.6.1)
```

```
#>    S4Arrays                1.12.0   2026-04-28 [1] https://bioc-release.r-universe.dev (R 4.6.0)
#>    S4Vectors             * 0.50.2   2026-08-23 [1] Bioconductor 3.23 (R 4.6.1)
#>    S7                      0.2.2    2026-04-22 [1] CRAN (R 4.6.1)
#>    scales                  1.4.0    2025-04-24 [1] CRAN (R 4.6.1)
#>    Seqinfo               * 1.2.0    2026-04-28 [1] https://bioc-release.r-universe.dev (R 4.6.0)
#>    sessioninfo             1.2.4    2026-06-04 [1] CRAN (R 4.6.1)
#>    ShortRead               1.70.0   2026-04-28 [1] https://bioc-release.r-universe.dev (R 4.6.0)
#>    SparseArray             1.12.2   2026-05-01 [1] https://bioc-release.r-universe.dev (R 4.6.0)
#>    stringi                 1.8.9    2026-08-04 [1] CRAN (R 4.6.1)
#>    stringr               * 1.6.0    2025-11-04 [1] CRAN (R 4.6.1)
#>    SummarizedExperiment    1.42.0   2026-04-28 [1] https://bioc-release.r-universe.dev (R 4.6.0)
#>    survival                3.8-6    2026-01-16 [2] CRAN (R 4.6.1)
#>    tibble                * 3.3.1    2026-01-11 [1] CRAN (R 4.6.1)
#>    tidyr                 * 1.3.2    2025-12-19 [1] CRAN (R 4.6.1)
#>    tidyselect              1.2.1    2024-03-11 [1] CRAN (R 4.6.1)
#>    tidyverse             * 2.0.0    2023-02-22 [1] CRAN (R 4.6.1)
#>    timechange              0.4.0    2026-01-29 [1] CRAN (R 4.6.1)
#>    tzdb                    0.5.0    2025-03-15 [1] CRAN (R 4.6.1)
#>    usethis               * 3.2.2    2026-09-10 [1] CRAN (R 4.6.1)
#>    vctrs                   0.7.3    2026-04-11 [1] CRAN (R 4.6.1)
#>    vegan                   2.7-6    2026-08-31 [1] CRAN (R 4.6.1)
#>    withr                   3.0.3    2026-06-19 [1] CRAN (R 4.6.1)
#>    xfun                    0.60     2026-07-09 [1] CRAN (R 4.6.1)
#>    xlsx                  * 0.6.5    2020-11-10 [1] CRAN (R 4.6.0)
#>    xlsxjars                0.9.0    2025-07-02 [1] CRAN (R 4.6.1)
#>    XVector               * 0.52.0   2026-04-28 [1] https://bioc-release.r-universe.dev (R 4.6.0)
#>    yaml                    2.3.12   2025-12-10 [1] CRAN (R 4.6.1)
#>
#>  [1] C:/Users/uax75/AppData/Local/R/win-library/4.6
#>  [2] C:/Program Files/R/R-4.6.1/library
#>
#>  * -- Packages attached to the search path.
#>  D -- DLL MD5 mismatch, broken installation.
#>
#> ------------------------------------------------------------------------------
```